ESGI - AL5 / T1

Apprentissage par renforcement

TP MDP : Snake

Processus de décision markoviens et Q-Learning

Rapport - Exercices 1 et 2

RUSESCU Daniel Alexandru
BAH Sulaiman
SEDJAI Fethi

Enseignant : Sylvain Gault
Date : 5 novembre 2026

Répartition du travail

Toutes les parties ont été réalisées ensemble par les trois membres du groupe : conception, code, tests, graphiques, analyse, rédaction et relecture. La mention « Travail collectif » associée à chaque question renvoie à cette répartition commune.

Code source : dépôt GitHub du groupe — github.com/Sulaimanzy/mdp-snake-alex-sulaiman-fethi

Sommaire

Reproduction des résultats

Exercice 1 — Jeu de base

- Question 1.1 — Classe ShortSnake et grille
- Question 1.2 — Position et direction initiales
- Question 1.3 — Position aléatoire du fruit
- Question 1.4 — Score
- Question 1.5 — Méthode move
- Question 1.6 — Franchissement d'un bord
- Question 1.7 — Fruit mangé
- Question 1.8 — Affichage __str__

Exercice 2 — Modélisation du MDP

- Question 2.1 — Informations constituant un état
- Question 2.2 — Nombre d'états pour une grille 10 × 10
- Question 2.3 — Grille N × M
- Question 2.4 — Actions du MDP
- Question 2.5 — Les actions dépendent-elles de l'état ?
- Question 2.6 — Récompenses du MDP
- Question 2.7 — Représentation graphique du MDP

Reproduction des résultats

- Python 3.14 (toute version ≥ 3.8 convient). Le jeu et le modèle MDP n'ont besoin d'aucune dépendance ; `matplotlib` sert aux figures, `markdown`, `pygments` et `pymupdf` à générer ce rapport.
- Depuis la racine du dépôt :

```
py -m pip install -r requirements.txt   # dépendances (figures et rapport)
py -m unittest discover tests           # tests du jeu et du modèle MDP
py ex1_demo.py                          # exercice 1 : partie scriptée
py ex1_demo.py --play                   # exercice 1 : jouer au clavier (z q s d + Entrée)
py ex2_mdp.py                           # exercice 2 : réponses calculées + figure du MDP
py rapport/build_pdf.py                 # régénère ce rapport en PDF
```

Fichiers :

Fichier	Contenu
`snake.py`	classe `ShortSnake` (mécanique du jeu)
`ex1_demo.py`	démonstration de l'exercice 1
`mdp.py`	modèle MDP du jeu : états, actions, dynamique p(s', r | s, a)
`ex2_mdp.py`	réponses de l'exercice 2 et figure `rapport/figures/ex2_mdp.png`
`tests/test_snake.py`	tests unitaires de `ShortSnake`
`tests/test_mdp.py`	tests du modèle MDP (comptages, probabilités, cohérence avec le jeu)
`rapport/rapport.md`	source de ce rapport
`rapport/build_pdf.py`	conversion du rapport en PDF (Markdown → HTML → PDF via Edge ou Chrome)

Exercice 1 — Jeu de base

Convention : les coordonnées sont des tuples `(ligne, colonne)`, `(0, 0)` étant le coin en haut à gauche. Les directions sont les chaînes `UP`, `DOWN`, `LEFT`, `RIGHT`.

```
EMPTY, SNAKE, FRUIT = 0, 1, 2

UP, DOWN, LEFT, RIGHT = "UP", "DOWN", "LEFT", "RIGHT"
DIRECTIONS = {UP: (-1, 0), DOWN: (1, 0), LEFT: (0, -1), RIGHT: (0, 1)}
OPPOSITE = {UP: DOWN, DOWN: UP, LEFT: RIGHT, RIGHT: LEFT}

WALL_PENALTY = -10
FRUIT_REWARD = 10
```

Question 1.1 — Classe `ShortSnake` et grille

Travail collectif

```
class ShortSnake:
    def __init__(self, size=10, seed=None):
        if isinstance(size, int):
            size = (size, size)
        self.n_rows, self.n_cols = size
        if self.n_rows < 1 or self.n_cols < 1 or self.n_rows * self.n_cols < 2:
            raise ValueError("la grille doit contenir au moins 2 cases")
        self.grid = [[EMPTY] * self.n_cols for _ in range(self.n_rows)]
        self.rng = random.Random(seed)
```

La taille est un entier (grille carrée) ou un couple `(lignes, colonnes)`. La grille est une liste de listes dont chaque case vaut `EMPTY`, `SNAKE` ou `FRUIT`. Il faut au moins 2 cases pour pouvoir placer le serpent et le fruit. Le générateur `rng` peut recevoir une graine pour rendre une partie reproductible.

Question 1.2 — Position et direction initiales

Travail collectif

```
        self.snake_pos = (self.n_rows // 2, self.n_cols // 2)
        self.snake_dir = RIGHT
        self._set_cell(self.snake_pos, SNAKE)

    def _set_cell(self, pos, value):
        row, col = pos
        self.grid[row][col] = value
```

Le serpent démarre au milieu de la grille (`(5, 5)` pour une grille 10 × 10) et part vers la droite.

Question 1.3 — Position aléatoire du fruit

Travail collectif

```
        self.fruit_pos = self._place_fruit()

    def _place_fruit(self):
        free_cells = [
            (row, col)
            for row in range(self.n_rows)
            for col in range(self.n_cols)
            if self.grid[row][col] == EMPTY
        ]
        pos = self.rng.choice(free_cells)
        self._set_cell(pos, FRUIT)
        return pos
```

Le fruit est tiré uniformément parmi les cases libres, il n'apparaît donc jamais sous le serpent.

Question 1.4 — Score

Travail collectif

```
        self.score = 0
```

Question 1.5 — Méthode `move`

Travail collectif

```
    def move(self, direction=None):
        if direction is not None:
            if direction not in DIRECTIONS:
                raise ValueError(f"direction inconnue : {direction!r}")
            if direction != OPPOSITE[self.snake_dir]:
                self.snake_dir = direction

        d_row, d_col = DIRECTIONS[self.snake_dir]
        row, col = self.snake_pos[0] + d_row, self.snake_pos[1] + d_col
        reward = 0
        ...
        self._set_cell(self.snake_pos, EMPTY)
        self.snake_pos = (row, col)
        self._set_cell(self.snake_pos, SNAKE)
        ...
        self.score += reward
        return reward
```

`None` garde la direction courante. Une demande de demi-tour est ignorée : le serpent continue dans sa direction actuelle. La méthode renvoie la récompense de l'étape, qui servira au Q-Learning (exercice 3).

Question 1.6 — Franchissement d'un bord

Travail collectif

```
        if not (0 <= row < self.n_rows and 0 <= col < self.n_cols):
            row %= self.n_rows
            col %= self.n_cols
            reward += WALL_PENALTY
```

Le modulo fait réapparaître le serpent du côté opposé (ligne `-1` → ligne `n_rows - 1`, colonne `n_cols` → colonne `0`), avec une pénalité de 10 points.

Question 1.7 — Fruit mangé

Travail collectif

```
        if self.snake_pos == self.fruit_pos:
            reward += FRUIT_REWARD
            self.fruit_pos = self._place_fruit()
```

Le test a lieu après le déplacement de la tête : la case du fruit est déjà marquée `SNAKE`, donc le nouveau fruit est tiré parmi les autres cases. Si le serpent franchit un bord et tombe sur le fruit, les deux récompenses s'additionnent (−10 + 10 = 0).

Question 1.8 — Affichage `__str__`

Travail collectif

```
HEAD_SYMBOL = {UP: "^", DOWN: "v", LEFT: "<", RIGHT: ">"}
CELL_SYMBOL = {EMPTY: ".", FRUIT: "*"}

    def __str__(self):
        border = "+" + "-" * self.n_cols + "+"
        lines = [border]
        for row in self.grid:
            cells = (
                HEAD_SYMBOL[self.snake_dir] if cell == SNAKE else CELL_SYMBOL[cell]
                for cell in row
            )
            lines.append("|" + "".join(cells) + "|")
        lines.append(border)
        lines.append(f"Score : {self.score}")
        return "\n".join(lines)
```

La tête est dessinée par une flèche qui indique sa direction. Extrait de `py ex1_demo.py` (grille 5 × 8, graine 42) :

```
Étape 5 : move(UP) -> récompense +10 (on mange le fruit : +10)
+--------+
|.*.....^|
|........|
|........|
|........|
|........|
+--------+
Score : 10

Étape 6 : move(UP) -> récompense -10 (bord du haut franchi : on réapparaît en bas, -10)
+--------+
|.*......|
|........|
|........|
|........|
|.......^|
+--------+
Score : 0
```

Les 17 tests de `tests/test_snake.py` vérifient chacun de ces comportements (position initiale, demi-tour ignoré, passage des quatre bords, fruit mangé, cohérence de la grille, affichage).

Exercice 2 — Modélisation du MDP

Le modèle est codé dans `mdp.py` ; `ex2_mdp.py` en déduit les réponses. Sortie de `py ex2_mdp.py` :

```
Q2.1  état = (tête, direction, fruit) = ((5, 5), 'RIGHT', (4, 9))
Q2.2  grille 10 x 10 : 39600 états (énumérés : 39600, atteignables : 39600)
Q2.3  |S| = 4·N·M·(N·M − 1) : {'3x3': 288, '4x5': 1520, '10x10': 39600, '20x20': 638400}
Q2.4  arguments de move : [None, 'UP', 'DOWN', 'LEFT', 'RIGHT']
      actions regroupées par effet dans ((5, 5), 'RIGHT', (4, 9)) : [[None, 'LEFT', 'RIGHT'],
['UP'], ['DOWN']]
Q2.5  A(s) si direction = UP    : ['UP', 'LEFT', 'RIGHT']
Q2.5  A(s) si direction = DOWN  : ['DOWN', 'LEFT', 'RIGHT']
Q2.5  A(s) si direction = LEFT  : ['UP', 'DOWN', 'LEFT']
Q2.5  A(s) si direction = RIGHT : ['UP', 'DOWN', 'RIGHT']
Q2.6  récompenses possibles : [-10, 0, 10]
      exemple bord  : [(1.0, ((5, 0), 'RIGHT', (0, 0)), -10)]
      exemple fruit : 99 états suivants, p = 1/99, r = 10
```

Question 2.1 — Informations constituant un état

Travail collectif

```python
def state_of(game):
    """État du MDP pour une partie en cours : (tête, direction, fruit)."""
    return (game.snake_pos, game.snake_dir, game.fruit_pos)
```

Un état contient tout ce qui détermine la suite du jeu (propriété de Markov) :

- la **position de la tête** `(ligne, colonne)` ;
- la **direction courante** : elle fixe l'effet de `None` (continuer tout droit) et le demi-tour interdit. Sans elle, la transition dépendrait des coups précédents ;
- la **position du fruit**, qui fixe la récompense des prochains déplacements.

Le score n'en fait pas partie : c'est la somme des récompenses déjà reçues, il n'influence pas la suite du jeu et rendrait l'ensemble des états infini. La grille non plus : elle se déduit de la tête et du fruit.

Question 2.2 — Nombre d'états pour une grille 10 × 10

Travail collectif

```python
def n_states(n_rows, n_cols):
    """|S| = N·M positions de tête × 4 directions × (N·M − 1) positions du fruit."""
    n_cells = n_rows * n_cols
    return n_cells * len(DIRECTIONS) * (n_cells - 1)


def all_states(size):
    """Énumère tous les états : le fruit n'est jamais sous la tête."""
    grid_cells = cells(size)
    return [
        (head, direction, fruit)
        for head in grid_cells
        for direction in DIRECTIONS
        for fruit in grid_cells
        if fruit != head
    ]
```

100 × 4 × 99 = 39 600 états. Le fruit n'est jamais sur la tête : dès que le serpent l'atteint, il est mangé et replacé sur une autre case. Le dénombrement est confirmé par l'énumération, et un parcours en largeur du graphe du MDP (`reachable_states`) montre que les 39 600 états sont tous atteignables depuis l'état initial.

Question 2.3 — Grille N × M

Travail collectif

Même fonction `n_states(n_rows, n_cols)` :

|S| = 4 · N · M · (N · M − 1)

Grille	3 × 3	4 × 5	10 × 10	20 × 20
États	288	1 520	39 600	638 400

Le nombre d'états croît comme le carré du nombre de cases. Le test `test_count_formula` vérifie la formule par énumération sur plusieurs tailles.

Question 2.4 — Actions du MDP

Travail collectif

```python
MOVE_ARGUMENTS = [None, UP, DOWN, LEFT, RIGHT]   # arguments acceptés par move


def actions(state):
    """A(s) : les quatre directions sauf le demi-tour (None équivaut à la direction courante)."""
    direction = state[1]
    return [d for d in DIRECTIONS if d != OPPOSITE[direction]]
```

`move` accepte cinq arguments, mais ils n'ont que **trois effets distincts** : continuer tout droit, tourner à gauche, tourner à droite. Dans l'état `((5, 5), RIGHT, …)`, `None`, `LEFT` (demi-tour ignoré) et `RIGHT` mènent au même état. On retient donc comme actions du MDP les trois directions qui ne sont pas un demi-tour.

Question 2.5 — Les actions dépendent-elles de l'état ?

Travail collectif

```
for direction in (UP, DOWN, LEFT, RIGHT):
    print(f"A(s) si direction = {direction:5} :", actions(((5, 5), direction, (0, 0))))
```

Oui : $\mathcal{A}$(s) = {UP, DOWN, LEFT, RIGHT} \ {opposé de la direction de s}. Seule la direction de l'état intervient, pas la position de la tête ni celle du fruit. On aurait pu modéliser des actions relatives (tout droit, gauche, droite), identiques dans tous les états ; leur effet absolu dépendrait alors de la direction. Nous gardons les directions absolues, qui correspondent directement aux arguments de `move`.

Question 2.6 — Récompenses du MDP

Travail collectif

```
def transitions(state, action, size):
    """p(s', r | s, a) sous forme de liste de (probabilité, état suivant, récompense)."""
    n_rows, n_cols = _size(size)
    (row, col), direction, fruit = state

    if action is not None and action != OPPOSITE[direction]:
        direction = action
    d_row, d_col = DIRECTIONS[direction]
    row, col = row + d_row, col + d_col

    reward = 0
    if not (0 <= row < n_rows and 0 <= col < n_cols):
        row, col = row % n_rows, col % n_cols
        reward += WALL_PENALTY
    head = (row, col)

    if head != fruit:  # déplacement déterministe
        return [(1.0, (head, direction, fruit), reward)]

    # Fruit mangé : le nouveau fruit est uniforme sur les N·M − 1 autres cases
    reward += FRUIT_REWARD
    free_cells = [cell for cell in cells(size) if cell != head]
    prob = 1 / len(free_cells)
    return [(prob, (head, direction, new_fruit), reward) for new_fruit in free_cells]
```

$\mathscr{R}$ = {−10, 0, +10} :

Situation	Récompense
fruit mangé	+10
bord franchi	−10
bord franchi et fruit mangé au même pas	−10 + 10 = 0
autre déplacement	0

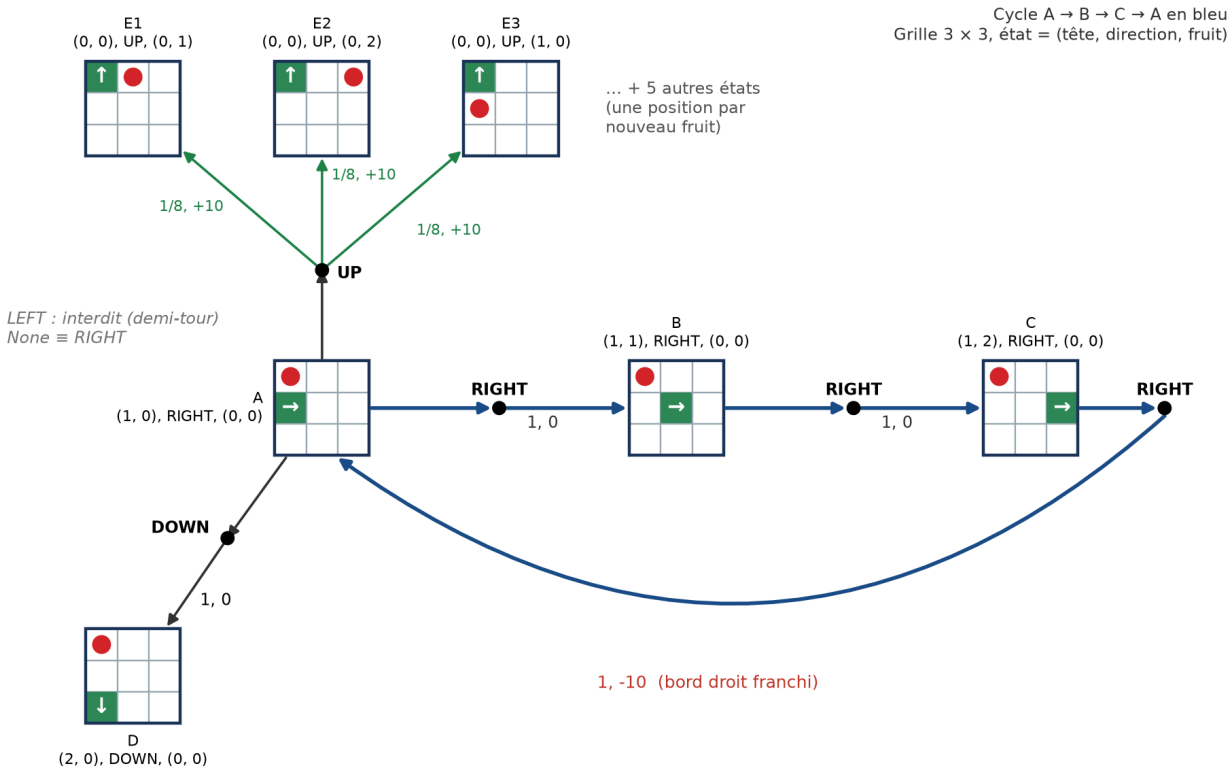
La fonction donne la dynamique complète p(s', r | s, a) du cours. Le déplacement est déterministe ; seul le placement du nouveau fruit est aléatoire, avec une probabilité 1 / (N·M − 1) pour chaque case libre. Il n'y a pas d'état final : c'est une tâche continue. Le test `test_model_matches_game` joue 2 000 coups aléatoires avec `ShortSnake` et vérifie que chaque résultat du jeu est bien prévu par `transitions`.

Question 2.7 — Représentation graphique du MDP

Travail collectif

```python
def draw_mdp_figure(path):
    size = (3, 3)

    # États du cycle, obtenus avec le modèle p(s', r | s, a)
    A = ((1, 0), RIGHT, (0, 0))
    [(p_ab, B, r_ab)] = transitions(A, RIGHT, size)
    [(p_bc, C, r_bc)] = transitions(B, RIGHT, size)
    [(p_ca, back, r_ca)] = transitions(C, RIGHT, size)
    assert back == A, "RIGHT trois fois depuis A doit revenir en A"
    [(p_ad, D, r_ad)] = transitions(A, DOWN, size)
    eaten = transitions(A, UP, size)  # fruit mangé : 8 états suivants
    ...  # dessin avec matplotlib (mini-grilles, points d'action, flèches « p, r »)
```



La figure reprend la notation du cours (robot aspirateur) sur une grille 3 × 3 : chaque état est dessiné par sa mini-grille (tête verte fléchée, fruit rouge), chaque point noir est une action, chaque flèche porte « probabilité, récompense ».

- **Cycle A → B → C → A** (en bleu) : en répétant `RIGHT` sur la ligne du milieu, le serpent traverse le bord droit et revient en A. Les récompenses du cycle sont 0, 0 et −10.
- **A —UP→** : la tête arrive sur le fruit (+10) ; le nouveau fruit est tiré parmi les 8 autres cases, d'où 8 états suivants de probabilité 1/8 (3 sont dessinés).
- **A —DOWN→ D** : déplacement déterministe sans récompense.
- **LEFT** est un demi-tour depuis A : il n'est pas dans $\mathcal{A}$(A), et `None` équivaut à `RIGHT`. Pour B et C, seule l'action du cycle est représentée.